



Práctica 4

DISPLAYS MULTIPLEXADOS

Sistemas en Chips

- ▶ **Prof. José Juan Pérez Pérez**
- ▶ **Grupo: 6CVI**
- ▶ **Equipo I:**
 - ▶ Ayala Fuentes Sunem Gizeht
 - ▶ Guevara Badillo Areli Alejandra
 - ▶ León Pérez May Carolina

Introducción

Multiplexación en Displays

El Problema

Controlar un solo display de 7 segmentos requiere 7 pines para los segmentos (a-g) y un pin común para encenderlo o apagarlo; un total de 8 pines. Para controlar 6 displays de esta manera, se necesitarían $6 \times 7 = 42$ pines de segmento, más 6 pines comunes, sumando **48 pines**. Esto es impráctico y excede la capacidad de la mayoría de los microcontroladores.

La Solución

La multiplexación resuelve esto mediante dos conceptos clave: buses compartidos y conmutación de alta velocidad.

1. **Bus de Datos Común:** Todos los segmentos *iguales* de todos los displays se conectan juntos a un único pin de salida del microcontrolador. Esto se repite para los 7 segmentos.
2. **Bus de Selección (Control):** Cada display conserva su pin "común" de forma independiente. Estos pines de control se conectan a pines de salida individuales del microcontrolador.

Usando esta configuración, el número total de pines necesarios se reduce drásticamente a **7 (datos) + 6 (selección) = 13 pines**.

El Proceso (Barrido)

El microcontrolador ejecuta un bucle de "barrido" a alta velocidad:

1. Pone el patrón del **carácter 1** en el Bus de Datos.
2. Activa **solo el display 1** a través del Bus de Selección (y se asegura que los otros 5 estén apagados).
3. Espera un breve instante (milisegundos o microsegundos).
4. Apaga el display 1.
5. Pone el patrón del **carácter 2** en el Bus de Datos.
6. Activa **solo el display 2...** y así sucesivamente para los 6 displays.

Si este barrido completo se repite lo suficientemente rápido, el ojo humano no puede percibir el parpadeo. Gracias al fenómeno de la **persistencia retiniana** (POV), el cerebro integra estas imágenes secuenciales y rápidas, percibiendo un mensaje sólido y estable, como si todos los displays estuvieran encendidos al mismo tiempo.

Macros en Lenguaje Ensamblador

Una macro es una "plantilla de texto" que permite agrupar una secuencia de instrucciones bajo un solo nombre. Cuando el ensamblador procesa el código, cada vez que encuentra el nombre de la macro, lo reemplaza por la secuencia completa de instrucciones que define.

El propósito principal de las macros es la **abstracción**, lo que conlleva dos beneficios clave:

1. **Legibilidad del Código:** Permiten ocultar secuencias de instrucciones complejas o repetitivas detrás de un nombre simple y descriptivo. Esto hace que el código sea más fácil de leer y entender a un nivel superior.
2. **Mantenibilidad:** Si una secuencia de instrucciones se usa 20 veces en un programa y necesita ser modificada, solo se necesita cambiar la definición de la macro en un lugar, en lugar de buscar y editar las 20 instancias.

Macro vs. Subrutina

Es crucial distinguir una macro de una subrutina.

Característica	Macro	Subrutina
Naturaleza	Una directiva del ensamblador.	Un bloque de código máquina en la memoria.
Ensamblado	El código se copia en cada lugar donde se invoca.	El código existe solo una vez en la memoria del programa.
Ejecución	No requiere saltos.	Requiere un salto a la dirección de la subrutina y un retorno.
Memoria (Flash)	Usa más memoria de programa, ya que el código se duplica en cada invocación.	Ahorra memoria de programa, ya que el código es reutilizado.
Velocidad	Más rápida. No hay "sobrecarga" por las instrucciones de salto y retorno.	Más lenta. Consume ciclos de CPU adicionales para los saltos y la gestión de la pila.
Uso de Pila	No usa la pila.	Usa la pila (stack) para almacenar la dirección de retorno.

¿Cuándo usar una Macro?

Se usan idealmente para secuencias de código muy cortas (2-5 instrucciones) que se repiten con frecuencia, donde la velocidad de ejecución es crítica y el costo de memoria adicional es aceptable.

Planteamiento del ejercicio

El objetivo de esta práctica es diseñar e implementar un sistema de visualización dinámica utilizando el microcontrolador ATmega8535. El sistema debe ser capaz de mostrar una cadena de caracteres alfanuméricos distribuida a lo largo de un conjunto de **seis (6) displays de siete segmentos**.

Dado que la conexión directa de seis displays excedería la cantidad de pines de E/S disponibles en el microcontrolador, se requiere la implementación de la técnica de **multiplexación en el tiempo** para el control eficiente de los recursos.

Desarrollo

Material

Para la implementación de la práctica, se requirieron los siguientes componentes y herramientas:

- Microcontrolador ATmega8535.
- 6 Displays de 7 Segmentos de Cátodo Común.
- Fuente de alimentación: 5V DC.
- 2 placa de pruebas.
- Cables de conexión.

Código

```

.include "m8535def.inc" ; Incluye el archivo de definiciones del microcontrolador

; Definición de constantes para los patrones de 7 segmentos (Cátodo Común)
.equ B = $7C ; Patrón para la letra 'b'
.equ I = $30 ; Patrón para la letra 'I'
.equ R = $50 ; Patrón para la letra 'r'
.equ A = $77 ; Patrón para la letra 'A'

; MACRO: ldb (Load Byte)
; Carga un valor inmediato en un registro de propósito general utilizando r16 como registro temporal.
.macro ldb
    ldi r16, @1 ; Carga el valor constante en el registro temporal r16
    mov @0, r16 ; Mueve el valor de r16 al registro destino (@0)
.endm

; Renombrado de registros para facilitar la lectura
.def col = r17 ; Registro para controlar la selección de columna (Display)
.def dato = r18 ; Registro para el dato (Patrón de segmentos)

; Inicialización del Puntero de Pila (Stack Pointer)
ldi dato, low(RAMEND)
out SPL, dato ; Carga la parte baja de la dirección final de la RAM
ldi dato, high(RAMEND)
out SPH, dato ; Carga la parte alta de la dirección final de la RAM

; Configuración de Puertos de E/S
ser dato ; Pone todos los bits de 'dato' en 1 ($FF)
out DDRB, dato ; Configura Puerto B como SALIDA (Bus de Datos)
out DDRC, dato ; Configura Puerto C como SALIDA (Bus de Control)

; Carga de los patrones del mensaje en los registros r0 - r5
; Se utiliza la macro definida anteriormente.
ldb r0, B ; Carga 'B' en r0
ldb r1, I ; Carga 'I' en r1
ldb r2, R ; Carga 'R' en r2
ldb r3, R ; Carga 'R' en r3
ldb r4, I ; Carga 'I' en r4
ldb r5, A ; Carga 'A' en r5

clr ZH ; Limpia el byte alto del puntero Z (siempre será 0)

dos:
clr ZL ; Reinicia el puntero Z a la dirección de memoria $0000
ldi col, 4 ; Inicializa el selector en el 3er bit (PC2) -> 00000100
; Esto asume que el primer display está en PC2.

uno:
; --- Activación del Display ---
com col ; Invierte los bits. Ej: 00000100 -> 11111011
out PORTC, col ; Envía el valor al Puerto C. El '0' activa el cátodo común.
com col ; Vuelve a invertir para recuperar el valor original (00000100)

; --- Envío del Dato ---
ld dato, Z+ ; Carga el dato apuntado por Z en 'dato' e incrementa Z
out PORTB, dato ; Envía el patrón de segmentos al Puerto B (enciende LEDs)

; --- Persistencia de Visión ---
rcall delay ; Espera un tiempo para que el ojo perciba el carácter

; --- Apagado y Preparación ---
out PORTB, ZH ; Envía 0 al Puerto B (apaga segmentos) para evitar "Ghosting"
lsl col ; Desplazamiento lógico a la izquierda (Shift Left)
; Mueve el bit activo al siguiente display (PC2->PC3->etc.)

; --- Verificación de fin de barrido ---
cpi col, $00 ; Compara si el bit "1" se desbordó del registro (llegó a 0)
brne uno ; Si no es 0, salta a 'uno' para el siguiente display
rjmp dos ; Si es 0 (terminaron los 6 displays), reinicia el ciclo completo

delay:
ldi R19, $0F
WLOOP0:
ldi R20, $2A
WLOOP1:
dec R20
brne WLOOP1
dec R19
brne WLOOP0
nop
ret

```

Ilustración 1. Código implementado.

Solución

El algoritmo opera de la siguiente manera:

Inicialización

Se configuran los puertos B y C como salidas digitales.

El Puerto B actúa como bus de datos (segmentos) y el Puerto C como bus de control (selección de display).

Definición de Caracteres

Mediante la macro ldb, se cargan los patrones hexadecimales correspondientes a las letras "B", "I", "R", "R", "I", "A" en los registros de trabajo.

Ciclo de Barrido (Loop Principal)

El programa entra en un bucle infinito donde activa secuencialmente cada uno de los 6 displays.

Utiliza una máscara de bits rotativa en el registro col para enviar un '0' lógico al cátodo común del display objetivo a través del PORTC.

Simultáneamente, envía el patrón del carácter correspondiente al PORTB.

Persistencia de Visión

Se invoca una subrutina de retardo (delay) calibrada para mantener el display encendido el tiempo suficiente para ser percibido por el ojo humano, pero lo suficientemente corto para permitir que el ciclo completo se repita a una frecuencia superior a 60Hz, creando la ilusión de una imagen estática y continua.

Circuito

La interfaz de hardware conecta el microcontrolador con los displays de la siguiente forma:

- **Bus de Datos:** Los pines PB0 a PB7 del microcontrolador se conectan en paralelo a los segmentos a a dp de todos los displays.
- **Bus de Control:** Los pines del Puerto C se conectan individualmente a los pines comunes de cada uno de los 6 displays, permitiendo la activación independiente de cada dígito.

Diagrama del circuito

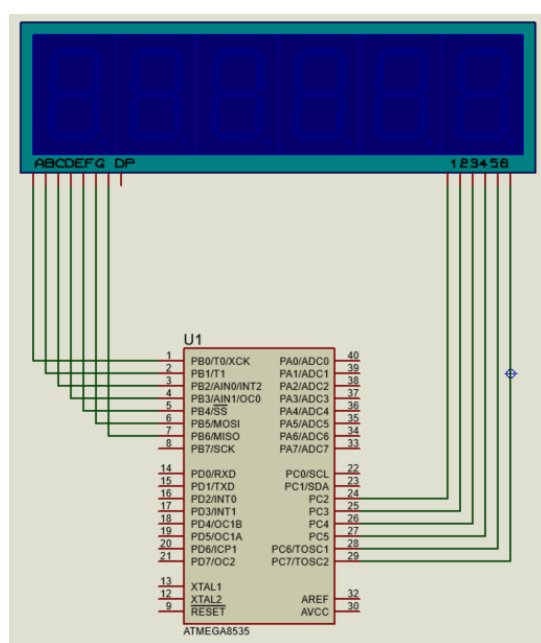


Ilustración 2. Esquema del circuito en proteus.

Simulación

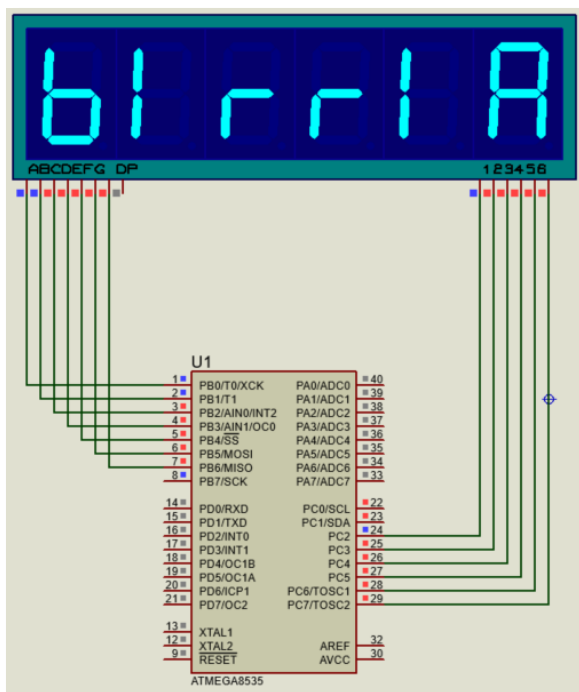


Ilustración 3. Simulación del funcionamiento en proteus.

Circuito Armado

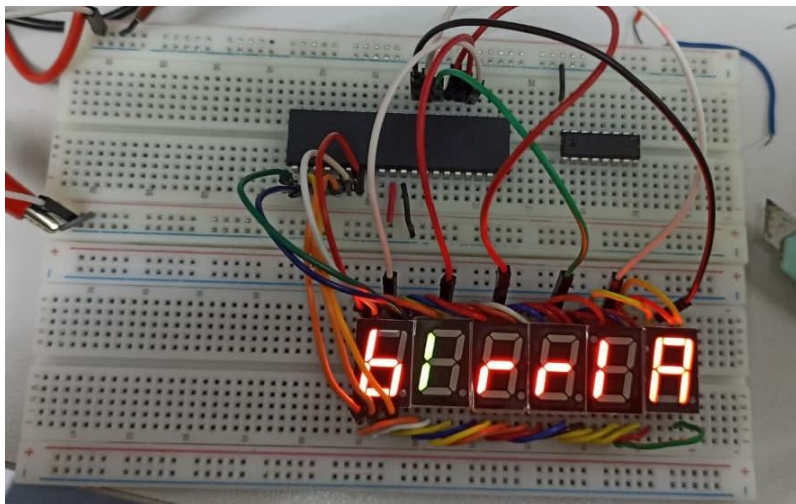


Ilustración 4. Armado físico del circuito funcionando.

Conclusiones

Ayala Fuentes Sunem Gizeht

Más allá de encender LEDs, esta práctica evidenció que el éxito de un sistema digital interactivo depende críticamente del manejo del tiempo. La visualización estable de la palabra 'BIRRIA' fue el resultado de calibrar con precisión la velocidad de procesamiento del CPU con la respuesta biológica del ojo humano (persistencia retiniana). Concluimos que en la programación de bajo nivel, la lógica booleana no es suficiente; es imperativo dominar los ciclos de reloj y las subrutinas de retardo (*delays*). Un desajuste en estos tiempos, o un error en la limpieza de los puertos (*ghosting*), arruinaría la ilusión óptica, enseñándonos que el control del 'cuándo' ocurre una instrucción es tan vital como el 'qué' hace esa instrucción.

Guevara Badillo Areli Alejandra

El desarrollo de esta práctica nos permitió comprender la estrecha relación que existe entre el diseño del circuito físico y la lógica de programación. Al enfrentarnos a la limitación de pines del microcontrolador ATmega8535, aprendimos que no es necesario escalar el hardware para controlar más dispositivos, sino que podemos utilizar soluciones de software como la multiplexación en el tiempo. Esta técnica, sumada al uso de macros en ensamblador para organizar las instrucciones, nos demostró que un sistema embebido eficiente es aquel donde el código asume la carga de trabajo compleja (el barrido rápido y la gestión de datos), permitiendo que el circuito físico se mantenga simple, económico y funcional, logrando visualizar el mensaje completo con un consumo mínimo de recursos.

León Pérez May Carolina

Aprendimos que trabajar en lenguaje Ensamblador no significa escribir código repetitivo o desordenado. La implementación de la macro `ldb` nos permitió simplificar tareas complejas en una sola instrucción, haciendo que nuestro programa fuera mucho más legible y fácil de modificar. Concluimos que el uso de herramientas de abstracción, como las macros, es indispensable para mantener un código profesional y estructurado, lo cual facilita enormemente la detección de errores durante el desarrollo del proyecto.

Referencias

- ▶ Multiplexación. (marzo 22, 2012). Recuperado de <https://synnick.blogspot.com/2012/03/multiplexacion.html>
- ▶ Multiplexación de Display 7 Segmentos. (n.d.). Recuperado de <https://es.slideshare.net/slideshow/multiplexacin-de-display-7-segmentos/63951590>
- ▶ Tema 7.3 Macros en ensamblador - Instituto Consorcio Clavijero. (n.d.). Recuperado de https://cursos.clavijero.edu.mx/cursos/082_ei/modulo7/contenido/tema7.3.html
- ▶ Jrking. (2016, April 21). MACRO LENGUAJE ENSAMBLADOR (ASM) y PROCEDIMIENTOS EN ENSAMBLADOR. Recuperado de <https://jrking95.wixsite.com/isc6semestre/single-post/2016/04/20/macro-lenguaje-ensamblador-asm-y-procedimientos-en-ensamblador>
- ▶ I. T. Informática de Gestión / Sistemas. (2008). Laboratorio de Estructura de Computadores. Recuperado de <http://atc2.aut.uah.es/~avicente/assignaturas/lec/pdf/pr6.pdf>